

Eviction Policies Part 1

Prepared By: Dr. D. P. Singh

LRU (Least Recently Used) Overview

- **LRU (Least Recently Used)** is a cache eviction strategy that removes the data that has not been **accessed for the longest time** when the cache becomes full.

Basic Idea

- Frequently accessed data stays in cache.
- Rarely accessed or old data gets removed first.
- Helps to keep **"hot" data available for faster access.**

How LRU works

Suppose the cache can store only 4 items.

Step	Operation	Cache State (Most Recent → Least Recent)
1	Add A	A
2	Add B	B, A
3	Add C	C, B, A
4	Add D	D, C, B, A
5	Access B	B, D, C, A
6	Add E (cache full)	E, B, D, C

When **E** is added, item **A** is evicted because it was the least recently used.

LRU in Redis

- Redis supports several eviction policies. The LRU-based policies are:

Policy	Description
<code>allkeys-lru</code>	Evict least recently used keys from all keys
<code>volatile-lru</code>	Evict least recently used keys having TTL only

Configuration Example

- When Redis reaches 1 GB memory:
 - New write arrives.
 - Redis finds least recently used keys.
 - Removes them.
 - Stores new data.

```
#maxmemory 1gb  
#maxmemory-policy allkeys-lru
```

Note: Redis does **not** implement a perfect LRU because tracking every access would be expensive.

Approximate LRU Algorithm

Redis uses an **approximate LRU algorithm**:

- Samples a small number of keys.
- Chooses the least recently used among sampled keys.
- Provides near-LRU performance with lower overhead.

LRU in Memcached

- Memcached uses **true LRU queues**.
- Each item belongs to an LRU list.

Process

- Item inserted → placed at top of LRU list.
- Item accessed → moved to top.
- Cache becomes full.
- Item at bottom (oldest unused) is evicted.

Advantages of LRU

- ✓ Simple and intuitive
- ✓ Keeps frequently accessed data in cache
- ✓ Improves cache hit ratio for many workloads
- ✓ Works well for web applications, APIs, and databases

Disadvantages of LRU

- ✗ Popular data can still be evicted if not accessed recently
- ✗ Sequential scans can pollute the cache
- ✗ Requires tracking access history
- ✗ Not optimal for all workloads

Cache Pollution

- Imagine a cache that can hold only 4 items.
- Initially:

Cache: A B C D

Suppose:

- A, B, C, and D are popular items accessed frequently.
- A user runs a report that reads 1000 records sequentially: E, F, G, H, I, J, ...

With LRU:

- Now all the frequently used items (A, B, C, D) are gone.
- After the report finishes, users request A again:
- The cache must fetch A from the database, causing slower performance.
- **The one-time scan data (E, F, G, H, ...) temporarily fills the cache even though it is unlikely to be reused.**
- **The cache becomes "polluted" with low-value data while valuable data is evicted.**

Real-World Example

Consider an e-commerce website:

- Product pages are stored in Redis/Memcached.
- Users frequently visit:
 - iPhone page
 - Laptop page
 - Headphones page
- These pages remain in cache because they are repeatedly accessed.
- A rarely viewed product page (e.g., an old accessory) may not be accessed for days and becomes the **least recently used** item. When memory is full, it gets evicted first.

Redis vs Memcached LRU

Feature	Redis	Memcached
LRU Type	Approximate LRU	True LRU
Memory Efficiency	Very High	High
Accuracy	Near-LRU	Exact LRU
Configuration	Multiple eviction policies	Built-in LRU
Suitable For	Large distributed caches	Simple caching workloads

Note: In practice, Redis's approximate LRU is extremely effective and scales well, while Memcached's LRU implementation is more precise but simpler in functionality.

Thank You